

Bypass de Détection Root Android avec Objection

Laboratoire d'Instrumentation Mobile

Cours : Sécurité des applications mobiles

Analyste : Omar Haouani

Outil : Objection (Frida-based)

APK cible : com.pwnsec.firestorm

Date : 20 mai 2026

Table des matières

1	Introduction	3
1.1	Objectifs	3
1.2	Prérequis	3
2	Environnement Technique	3
3	Phase 1 : Vérification de l'Environnement Frida	3
4	Phase 2 : Installation d'Objection	5
5	Phase 3 : Lancement d'Objection sur l'Application Cible	5
6	Phase 4 : Recherche des Méthodes de Détection	6
7	Phase 5 : Bypass des Détections de Root	6
7.1	Explication de "android root disable"	7
7.2	Explication de "android sslpinning disable"	7
8	Phase 6 : Validation du Bypass	7
9	Phase 7 : Automatisation - Injection au Démarrage	7
10	Phase 8 : Gestion des Checks Natifs (C/C++)	8
11	Résumé des Commandes Objection	8
12	Schéma d'Exploitation	9
13	Recommandations pour les Développeurs	9
14	Checklist	9
15	Conclusion	10

1 Introduction

Ce laboratoire porte sur l'utilisation d'**Objection**, un outil d'instrumentation mobile basé sur Frida, pour contourner les mécanismes de détection de root dans une application Android. Objection simplifie le processus de hooking en fournissant des commandes prêtes à l'emploi.

1.1 Objectifs

- Installer et configurer Objection
- Se connecter à une application cible
- Utiliser la commande `android root disable`
- Valider le bypass des détections de root
- Automatiser l'injection au démarrage
- Gérer les cas de checks natifs (C/C++)

1.2 Prérequis

- Python 3.8+ et pip
- ADB (Android Platform Tools)
- Appareil Android 8.0+ rooté
- Frida et frida-server (versions alignées)

2 Environnement Technique

Composant	Spécification
Appareil/Émulateur	Android 16 (émulateur)
Frida Server	16.x
Objection	v1.12.4
APK cible	com.pwnsec.firestorm

3 Phase 1 : Vérification de l'Environnement Frida

Avant de commencer, on vérifie que Frida est opérationnel sur l'appareil cible.

```

C:\Windows\System32>frida-ps -U
  PID  Name
-----
14318  Calendar
14153  Chrome
14173  Clock
12507  ConverterTabsJava
11290  FragmentsLab
  1966  Google
13718  JNIDemo
  4116  Messages
14458  Phone
  1388  SIM Toolkit
14073  Settings
  4571  abb
   581  adbd
   505  android.hardware.audio.service
   524  android.hardware.authsecret-service.example
1158  android.hardware.biometrics.fingerprint-service.ranchu
   520  android.hardware.bluetooth-service.default
   509  android.hardware.camera.provider.ranchu
   510  android.hardware.camera.provider@2.7-service-google
   532  android.hardware.cas-service.example
   539  android.hardware.contexthub-service.example
   565  android.hardware.drm-service.widevine
   275  android.hardware.gatekeeper-service.nonsecure
1270  android.hardware.gnss-service.ranchu
   511  android.hardware.graphics allocator-service.ranchu
   521  android.hardware.graphics.composer3-service.ranchu
   514  android.hardware.health-service.example
   522  android.hardware.identity-service.example
   523  android.hardware.lights-service.example
   515  android.hardware.media.c2@1.0-service-goldfish
   548  android.hardware.neuralnetworks-service-sample-all
   549  android.hardware.neuralnetworks-service-sample-limited
   550  android.hardware.neuralnetworks-shim-service-sample
   551  android.hardware.power-service.example
   555  android.hardware.power.stats-service.example
   516  android.hardware.radio-service.ranchu

```

FIGURE 1 – Liste des processus avec frida-ps -U

Résultat : L'émulateur répond correctement et les processus Android sont listés (Calendar, Chrome, Clock, Settings, etc.).

```
frida-ps -U
```


6 Phase 4 : Recherche des Méthodes de Détection

```
com.pwnsec.firestorm (run) on (Android: 16) [usb] # android hooking search classes
root
com.pwnsec.firestorm (run) on (Android: 16) [usb] # android hooking search methods
isRoot
android.inputmethodservice.InputMethodService$Insets.-$$$Nest$mdumpDebug
android.inputmethodservice.InputMethodService$Insets.$init
android.inputmethodservice.InputMethodService$Insets.dumpDebug
android.inputmethodservice.IInputMethodSessionWrapper.-$$$Nest$fgetmContext
android.inputmethodservice.IInputMethodSessionWrapper.$init
android.inputmethodservice.IInputMethodSessionWrapper.doFinishSession
android.inputmethodservice.IInputMethodSessionWrapper.appPrivateCommand
android.inputmethodservice.IInputMethodSessionWrapper.displayCompletions
android.inputmethodservice.IInputMethodSessionWrapper.executeMessage
android.inputmethodservice.IInputMethodSessionWrapper.finishInput
android.inputmethodservice.IInputMethodSessionWrapper.finishSession
android.inputmethodservice.IInputMethodSessionWrapper.getInternalInputMethodSession
android.inputmethodservice.IInputMethodSessionWrapper.invalidateInput
android.inputmethodservice.IInputMethodSessionWrapper.removeImeSurface
android.inputmethodservice.IInputMethodSessionWrapper.updateCursor
android.inputmethodservice.IInputMethodSessionWrapper.updateCursorAnchorInfo
android.inputmethodservice.IInputMethodSessionWrapper.updateExtractedText
android.inputmethodservice.IInputMethodSessionWrapper.updateSelection
android.inputmethodservice.IInputMethodSessionWrapper.viewClicked
android.inputmethodservice.IInputMethodSessionWrapper$ImeInputEventReceiver.$init
android.inputmethodservice.IInputMethodSessionWrapper$ImeInputEventReceiver.hasKeyModifiers
android.inputmethodservice.IInputMethodSessionWrapper$ImeInputEventReceiver.needsVerification
android.inputmethodservice.IInputMethodSessionWrapper$ImeInputEventReceiver.finishedEvent
android.inputmethodservice.IInputMethodSessionWrapper$ImeInputEventReceiver.onInputEvent
android.inputmethodservice.InputMethodService$InputMethodSessionImpl.$init
android.inputmethodservice.InputMethodService$InputMethodSessionImpl.appPrivateCommand
android.inputmethodservice.InputMethodService$InputMethodSessionImpl.displayCompletions
android.inputmethodservice.InputMethodService$InputMethodSessionImpl.finishInput
android.inputmethodservice.InputMethodService$InputMethodSessionImpl.invalidateInputInternal
```

FIGURE 3 – Recherche des classes et méthodes liées au root

Commandes exécutées :

```
android hooking search classes root
android hooking search methods isRoot
```

Objectif : Identifier les classes et méthodes utilisées par l'application pour détecter un environnement rooté.

7 Phase 5 : Bypass des Détections de Root

```
com.pwnsec.firestorm (run) on (Android: 16) [usb] # android root disable
(agent) Registering job 288511. Name: root-detection-disable
com.pwnsec.firestorm (run) on (Android: 16) [usb] # android sslpinning disable
(agent) Custom TrustManager ready, overriding SSLContext.init()
(agent) Found com.android.org.conscrypt.TrustManagerImpl, overriding TrustManagerImpl.verifyChain()
(agent) Found com.android.org.conscrypt.TrustManagerImpl, overriding TrustManagerImpl.checkTrustedRecursive()
(agent) Registering job 224455. Name: android-sslpinning-disable
com.pwnsec.firestorm (run) on (Android: 16) [usb] # android hooking search classes
root
```

FIGURE 4 – Activation des modules root disable et sslpinning disable

Commandes exécutées :

```
android root disable
android sslpinning disable
```

7.1 Explication de "android root disable"

La commande `android root disable` active automatiquement plusieurs hooks pour contourner les détections de root courantes :

- Hook de `File.exists` sur les chemins suspects (`/system/bin/su`, `/system/xbin/su`, etc.)
- Override de `Build.TAGS` pour remplacer "test-keys" par "release-keys"
- Hook de `Runtime.exec` pour bloquer les tentatives d'exécution de commandes shell
- Hook de `System.getenv` pour masquer les chemins contenant "su"

7.2 Explication de "android sslpinning disable"

La commande `android sslpinning disable` désactive le certificate pinning :

- Remplace les `TrustManager` par des versions personnalisées
- Override de `TrustManagerImpl.verifyChain()`
- Permet l'interception du trafic HTTPS via proxy (Burp, etc.)

8 Phase 6 : Validation du Bypass

Après exécution des commandes, on observe :

1. L'application ne détecte plus l'environnement rooté
2. Les messages "Root detected" disparaissent
3. Toutes les fonctionnalités de l'application deviennent accessibles

Vérification	Statut
Détection root	Bypassée
SSL Pinning	Désactivé
Fonctionnalités	Accessibles

9 Phase 7 : Automatisation - Injection au Démarrage

Pour automatiser le processus, on peut lancer Objection directement avec les commandes :

```
objection -g com.pwnsec.firestorm explore --startup-command "
  android root disable; android sslpinning disable"
```

Ou créer un script :

```
# bypass_root.obj
android root disable
android sslpinning disable
```

```
objection -g com.pwnsec.firestorm explore --startup-script
bypass_root.obj
```

10 Phase 8 : Gestion des Checks Natifs (C/C++)

Si l'application utilise des vérifications natives dans des bibliothèques .so, Objection propose des commandes supplémentaires :

```
android native hooking list exports
android native hooking watch --method name_of_function
```

Pour un bypass natif plus avancé, on peut utiliser Frida directement :

```
Java.perform(function() {
    var nativeFunc = Module.findExportByName("libnative.so", "
        check_root");
    Interceptor.replace(nativeFunc, new NativeCallback(function() {
        return 0; // false
    }, 'int', []));
});
```

11 Résumé des Commandes Objection

Commande	Fonction
objection -g APP start	Attacher à une application en cours
objection -n APP start	Démarrer et attacher à l'application
android root disable	Bypasser les détections de root
android sslpinning disable	Désactiver le certificate pinning
android hooking search classes KEYWORD	Rechercher des classes contenant KEYWORD
android hooking search methods KEYWORD	Rechercher des méthodes contenant KEYWORD
android hooking list classes	Lister toutes les classes
android hooking watch class METHOD	Surveiller l'exécution d'une méthode

12 Schéma d'Exploitation

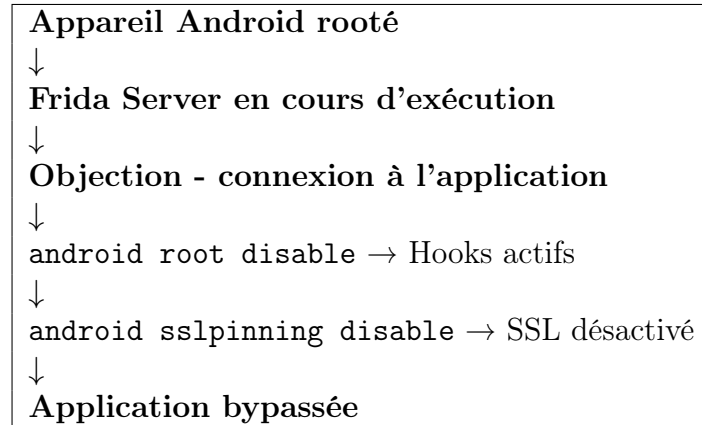


FIGURE 5 – Flux d'exploitation avec Objection

13 Recommandations pour les Développeurs

Pour se protéger contre les bypass via Objection/Frida :

1. Détecter Frida/objection (ports 27042, processus "frida")
2. Utiliser des vérifications en code natif (C/C++)
3. Ajouter du timing sur les vérifications (pas toutes au démarrage)
4. Vérifier l'intégrité du code (hash, signature)
5. Implémenter la détection de hooks (vérification des méthodes natives)
6. Utiliser l'obfuscation et la virtualisation de code

14 Checklist

- x Python et pip installés
- x ADB installé et fonctionnel
- x Appareil Android rooté et débogage USB activé
- x Frida Server installé sur l'appareil
- x Objection installé sur le PC
- x Connexion à l'application cible réussie
- x Commande `android root disable` exécutée
- x Bypass validé
- x Automatisation testée

15 Conclusion

Ce laboratoire a démontré :

1. La simplicité d'Objection pour le bypass des détections de root
2. L'efficacité de la commande `android root disable`
3. La complémentarité avec `android sslpinning disable`
4. La possibilité d'automatiser l'injection au démarrage

Objection est un outil puissant qui simplifie considérablement le travail d'instrumentation mobile, rendant accessible le bypass de protections à des débutants tout en restant utile pour les experts.

16 Ressources

- [Objection sur GitHub](#)
- [Documentation Frida](#)
- [OWASP MASVS - Anti-tampering](#)

Document réalisé dans le cadre du cours de Sécurité des applications mobiles

Omar Haouani - 20 mai 2026